



MICRO ARCHITECTURAL DESIGN OF DIGITAL SYSTEMS

Design and FPGA Implementation of Euclidean GCD Algorithm using 5 Stage Pipelined Processor

6th semester 2025-26

Project Report-Team 1

Guide- Dr. Saroja V S

SRN	Roll No.	Name
01FE23BEC203	440	VIVEK MIRAJE
01FE23BEC206	442	VAISHNAVI GAONKAR
01FE23BEC223	458	NAMAN KARADENNAVAR
01FE23BEC377	473	RAHUL HIREMATH

1. Problem Statement

Design and implement a hardware GCD processor on a Spartan-6 FPGA using a 5-stage pipelined MIPS32 architecture in Verilog. The processor executes hand-assembled GCD programs based on subtraction-based and modulus-based Euclidean algorithms, accepts 8-bit inputs via DIP switches, and displays the computed result on a 16×2 LCD and on 8 output LEDs(8 bit output).

2. Abstract

This report presents the FPGA implementation of a GCD processor using a 5-stage pipelined MIPS32 architecture on a Spartan-6 XC6SLX9. Two Euclidean algorithms — subtraction-based and modulus-based — are implemented as MIPS32 assembly programs. The design was first verified through simulation before deployment on the FPGA. Inputs are given via DIP switches, intermediate values are shown on LEDs during computation, and the final GCD is displayed on a 16×2 LCD after the processor halts.

3. Introduction

The Greatest Common Divisor (GCD) of two integers is the largest number that divides both without a remainder. It is widely used in cryptography, signal processing, and arithmetic simplification.

This project implements a pipelined MIPS32 processor in Verilog that computes GCD using two approaches — subtraction-based and modulus-based — and runs it on an FPGA.

Objectives:

- Implement a 5-stage pipelined MIPS32 processor in Verilog
- Code both GCD algorithms as MIPS32 assembly
- Confirm the functionality through simulation
- Deploy it on Spartan-6 FPGA with switch input and LED/LCD output

4. Background & Theory

4.1 Subtraction-Based Euclidean Algorithm

Repeatedly subtract the smaller number from the larger until both are equal. The result is the GCD.

```
while a ≠ b:
    if a > b : a = a - b
    else    : b = b - a
GCD = a
```

4.2 Modulus-Based Euclidean Algorithm

Replace the larger number with the remainder when divided by the smaller, until the remainder is zero. Converges faster than the subtraction-based approach for inputs with a large difference.

```

while b ≠ 0:
    r = a mod b
    a = b
    b = r
GCD = a

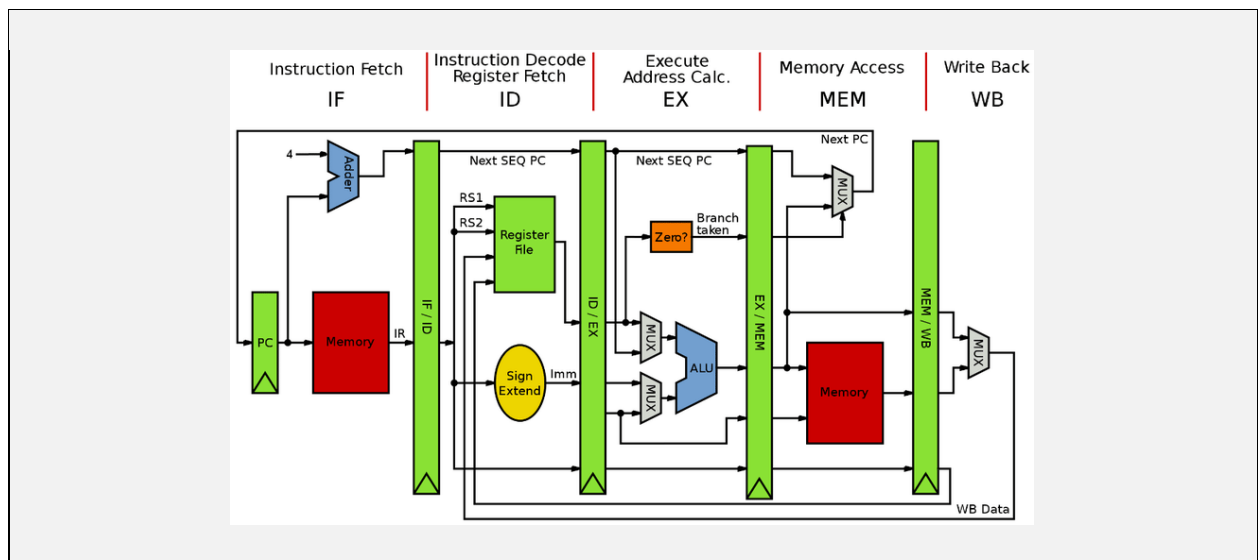
```

4.3 Instruction Set Used

Opcode	Operation
SUB	Register subtraction
SLT	Set 1 if $r_s < r_t$, else 0
ADDI	Add immediate value
BEQZ	Branch if register == 0
BNEQZ	Branch if register ≠ 0
HLT	Stop execution

4.4 5-Stage Pipeline

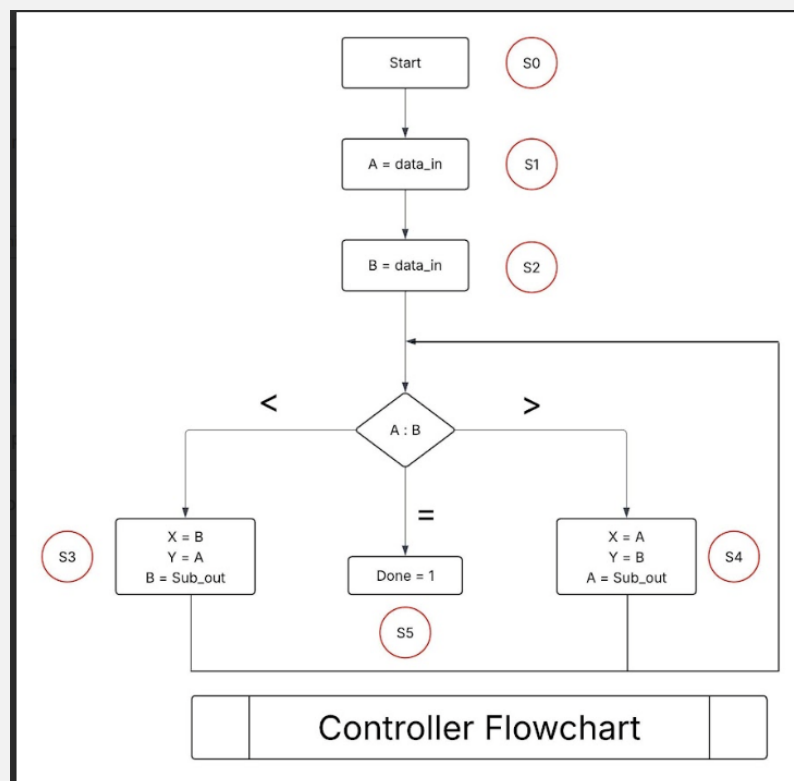
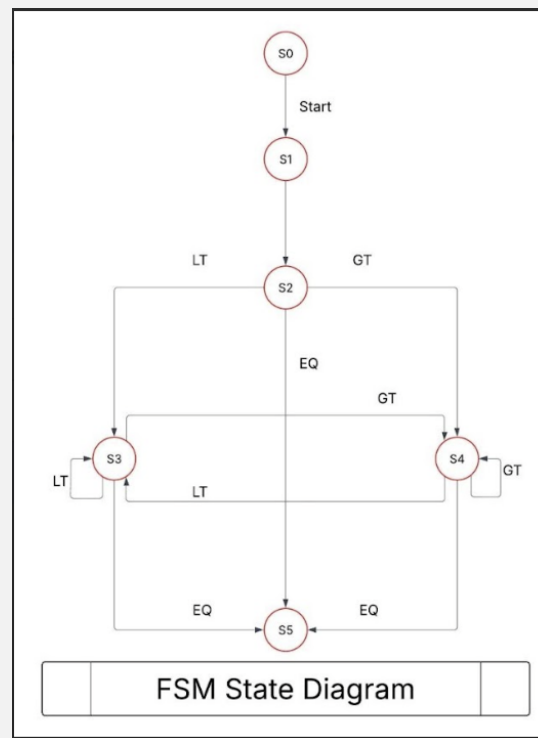
Instructions are broken into 5 stages — Fetch, Decode, Execute, Memory, and Write Back — so multiple instructions overlap in execution, improving throughput.

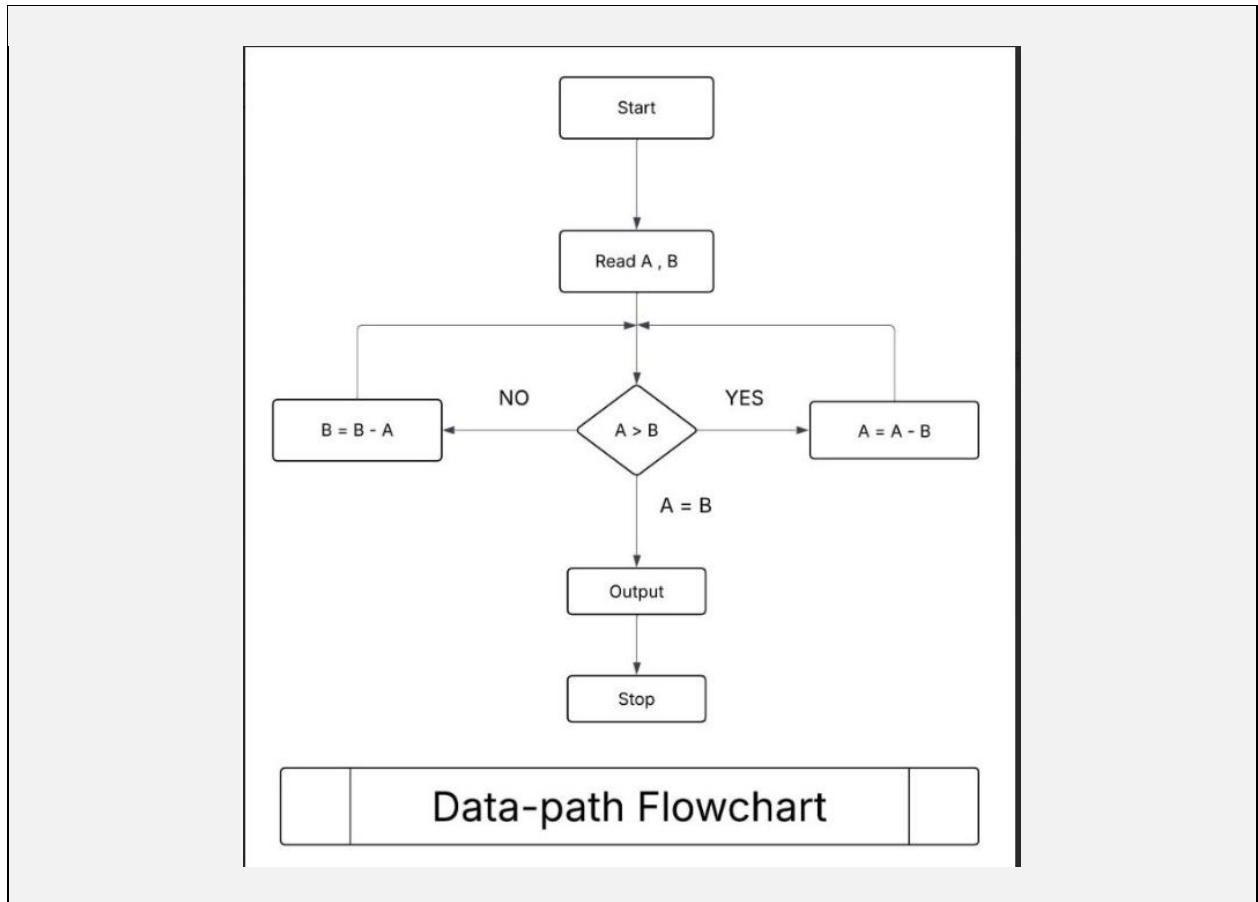


- **IF** — Fetches the instruction from memory using the Program Counter
- **ID** — Decodes the instruction and reads source registers
- **EX** — Performs the ALU operation or computes branch target
- **MEM** — Reads or writes data memory for load/store instructions
- **WB** — Writes the result back to the register file

5. Algorithm Implementation

5.1 Subtraction-Based GCD





Register Map

Register	Role
R0	Always zero
R1	a
R2	b
R3	Temp — holds a-b or SLT result

Assembly Program

```

; -----
; Subtraction-Based GCD
; -----

    NOP                ; PC 2 - pipeline padding

OUTER: SUB R3, R1, R2    ; PC 3 - R3 = a - b
    NOP                ; PC 4 - data hazard
    NOP                ; PC 5 - data hazard
    BEQZ R3, DONE        ; PC 6 - if a == b, GCD found
    NOP                ; PC 7 - branch delay
    NOP                ; PC 8 - branch delay

    SLT R3, R1, R2        ; PC 9 - R3 = 1 if a < b
    NOP                ; PC 10 - data hazard
    NOP                ; PC 11 - data hazard
    BNEQZ R3, B_GREATER    ; PC 12 - if a < b, jump
    NOP                ; PC 13 - branch delay
    NOP                ; PC 14 - branch delay

A_GREATER:
    SUB R1, R1, R2        ; PC 15 - a = a - b
    NOP                ; PC 16 - data hazard
    NOP                ; PC 17 - data hazard
    BEQZ R0, OUTER        ; PC 18 - unconditional loop
    NOP                ; PC 19 - branch delay
    NOP                ; PC 20 - branch delay

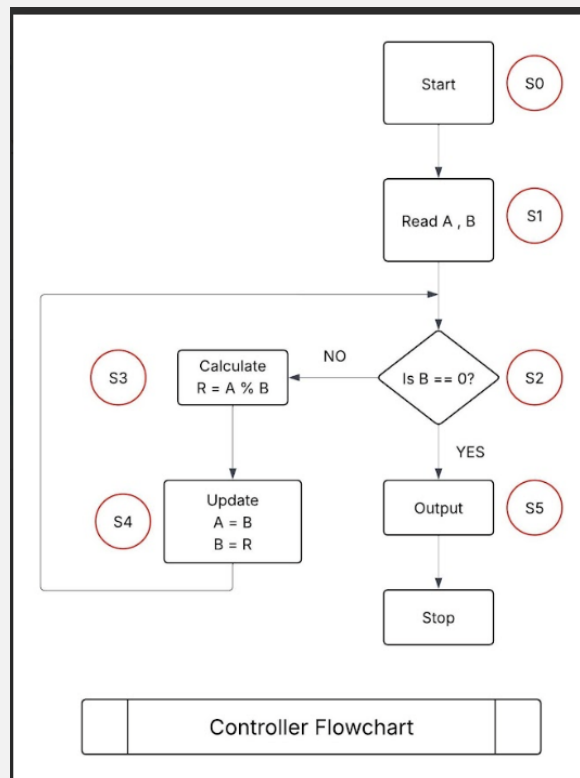
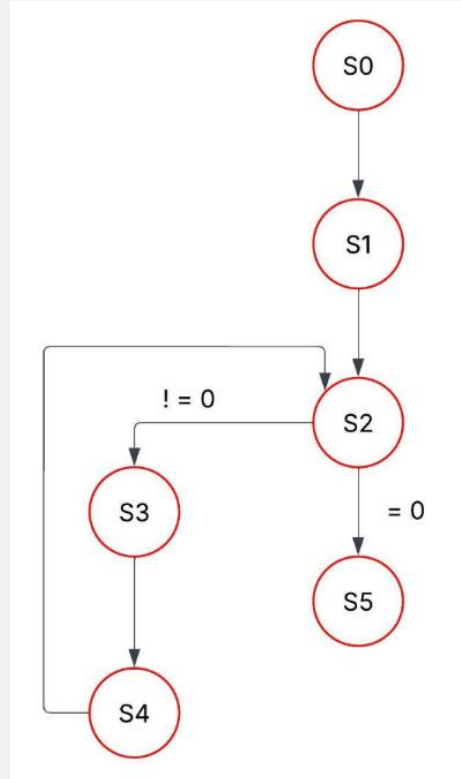
B_GREATER:
    SUB R2, R2, R1        ; PC 21 - b = b - a
    NOP                ; PC 22 - data hazard
    NOP                ; PC 23 - data hazard
    BEQZ R0, OUTER        ; PC 24 - unconditional loop
    NOP                ; PC 25 - branch delay
    NOP                ; PC 26 - branch delay

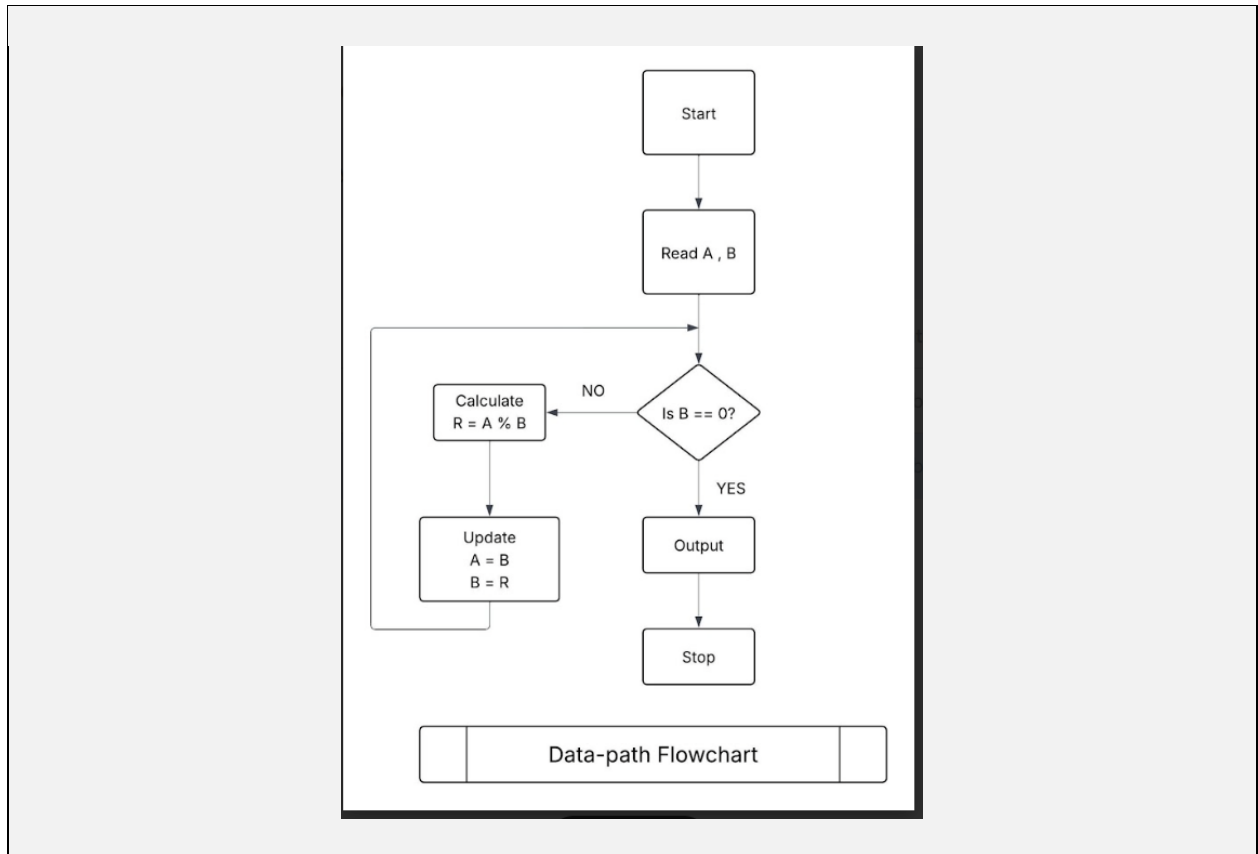
DONE: HLT                ; PC 27 - GCD = R1

```

5.2 Modulus-Based GCD

FSM STATE DIAGRAM





Register Map

Register	Role
R0	Always zero
R1	a / GCD result
R2	b — divisor
R3	r — remainder

Assembly Program

```

; -----
; Modulus-Based GCD
; -----

    NOP                ; PC 2 - pipeline padding

OUTER: BEQZ R2, DONE    ; PC 3 - if b == 0, GCD found
    NOP                ; PC 4 - branch delay
    NOP                ; PC 5 - branch delay

    ADDI R3, R1, 0      ; PC 6 - R3 = a (copy into remainder)
    NOP                ; PC 7 - data hazard
    NOP                ; PC 8 - data hazard

MOD_LOOP:
    SLT R1, R3, R2      ; PC 9 - R1 = 1 if remainder < divisor
    NOP                ; PC 10 - data hazard
    BNEQZ R1, UPDATE    ; PC 11 - if R3 < R2, mod done
    NOP                ; PC 12 - branch delay
    NOP                ; PC 13 - branch delay

    SUB R3, R3, R2      ; PC 14 - R3 -= R2
    NOP                ; PC 15 - data hazard
    BEQZ R0, MOD_LOOP   ; PC 16 - unconditional inner loop
    NOP                ; PC 17 - branch delay
    NOP                ; PC 18 - branch delay

UPDATE:
    ADDI R1, R2, 0      ; PC 19 - R1 = b (new a)
    NOP                ; PC 20 - data hazard
    ADDI R2, R3, 0      ; PC 21 - R2 = r (new b)
    NOP                ; PC 22 - data hazard
    NOP                ; PC 23 - data hazard
    BEQZ R0, OUTER      ; PC 24 - unconditional outer loop
    NOP                ; PC 25 - branch delay
    NOP                ; PC 26 - branch delay

DONE: HLT              ; PC 27 - GCD = R1

```

5.3 Comparison

Parameter	Subtraction-Based	Modulus-Based
Core operation	Subtract smaller from larger	Subtract divisor until remainder < divisor
Iterations	Many for large input difference	Few — faster convergence
Program size	28 instructions	28 instructions
Register usage	R0–R3	R0–R3

6. Pipeline Design

The processor uses a standard 5-stage MIPS pipeline — IF, ID, EX, MEM, WB. All five stages are implemented in a single always block using a phase flag that alternates every clock cycle.

- Phase 0 — Write Back + Execute + Instruction Fetch
- Phase 1 — Instruction Decode + Memory Access

6.1 Hazard Handling

Since the pipeline has no forwarding logic, data hazards are resolved by inserting 2 NOPs after every register-writing instruction, giving the write-back stage time to complete before the result is read.

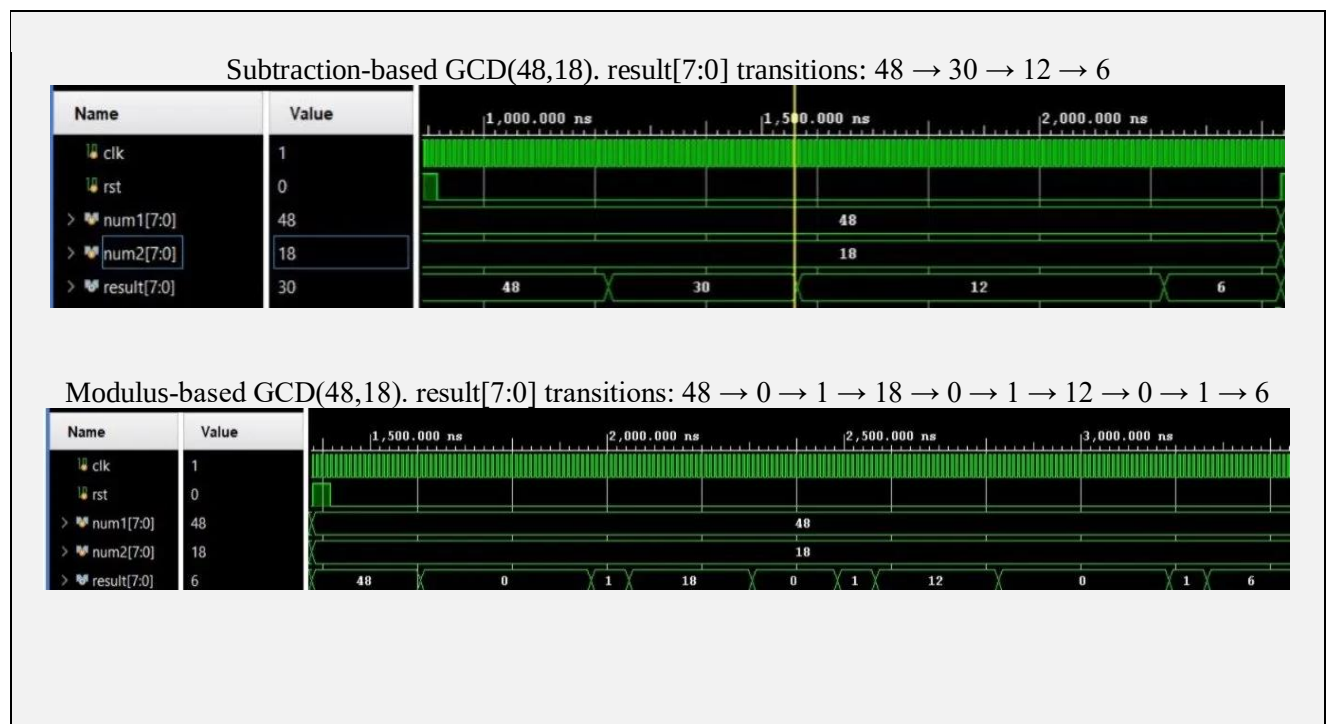
6.2 Branch Handling

Branch condition is computed in the EX stage. If the branch is taken, the PC is updated to the branch target and a TAKEN_BRANCH flag suppresses write-back of the two wrongly fetched instructions.

6.3 Simulation & Verification

Before programming onto the FPGA, the design was simulated in Xilinx ISE using a testbench. Trial inputs were applied and the waveforms were observed to verify correct pipeline operation — checking that register values updated at the right stages, branches resolved correctly. The final value in R1 was also confirmed through console output during simulation. Only after successful simulation was the design synthesized and deployed on the Spartan-6 board.

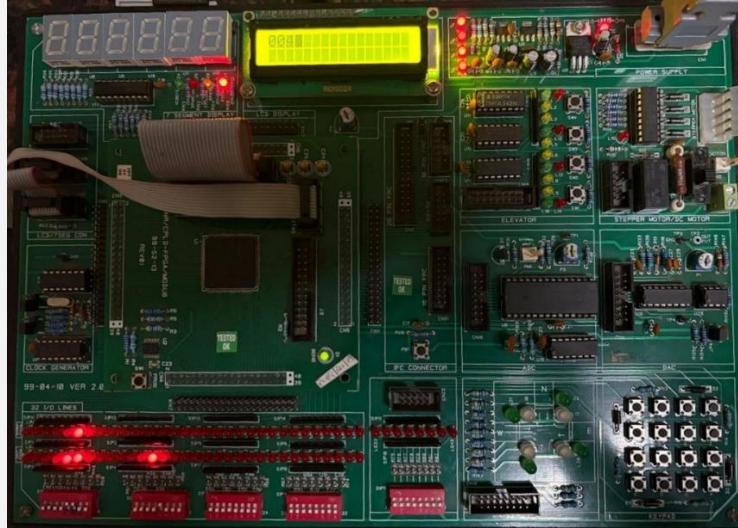
Timing Diagrams(Waveforms)



7. Results

Following successful simulation, the design was synthesized and programmed onto the Spartan-6 XC6SLX9 board. Inputs are set using DIP switches. During execution, R1 updates are visible on the LED array at every pipeline clock step. When HLT executes and halted goes high, the final GCD appears on the 16×2 LCD.

FPGA board displaying $\text{GCD}(12, 8) = 4$ on both the LED array and LCD



The processor was initially verified at the 10 MHz board clock. Once functional, the pipeline clock was intentionally slowed down to 1 Hz using the on-board clock divider so that the step-by-step updates to R1 are visible on the LED array, allowing the algorithm's progress to be observed in real time.

Test Cases

Test	Input A	Input B	Expected GCD	Observed GCD
1	48	18	6	6
2	100	75	25	25
3	7	13	1	1
4	255	85	85	85
5	12	12	12	12

8. Conclusion

A 5-stage pipelined MIPS32 GCD processor was successfully implemented on a Spartan-6 FPGA. Both subtraction-based and modulus-based Euclidean algorithms were hand-assembled into MIPS32 machine code and verified on hardware. The modulus-based approach converges faster for inputs with a large difference, while the subtraction-based approach is simpler in control flow. NOP insertion handles data hazards and branch flushing handles control hazards. Results were verified through LED monitoring during execution and LCD display after halt.

Future Scope

- Add forwarding and stall logic to eliminate NOPs
- Extend to 16-bit inputs